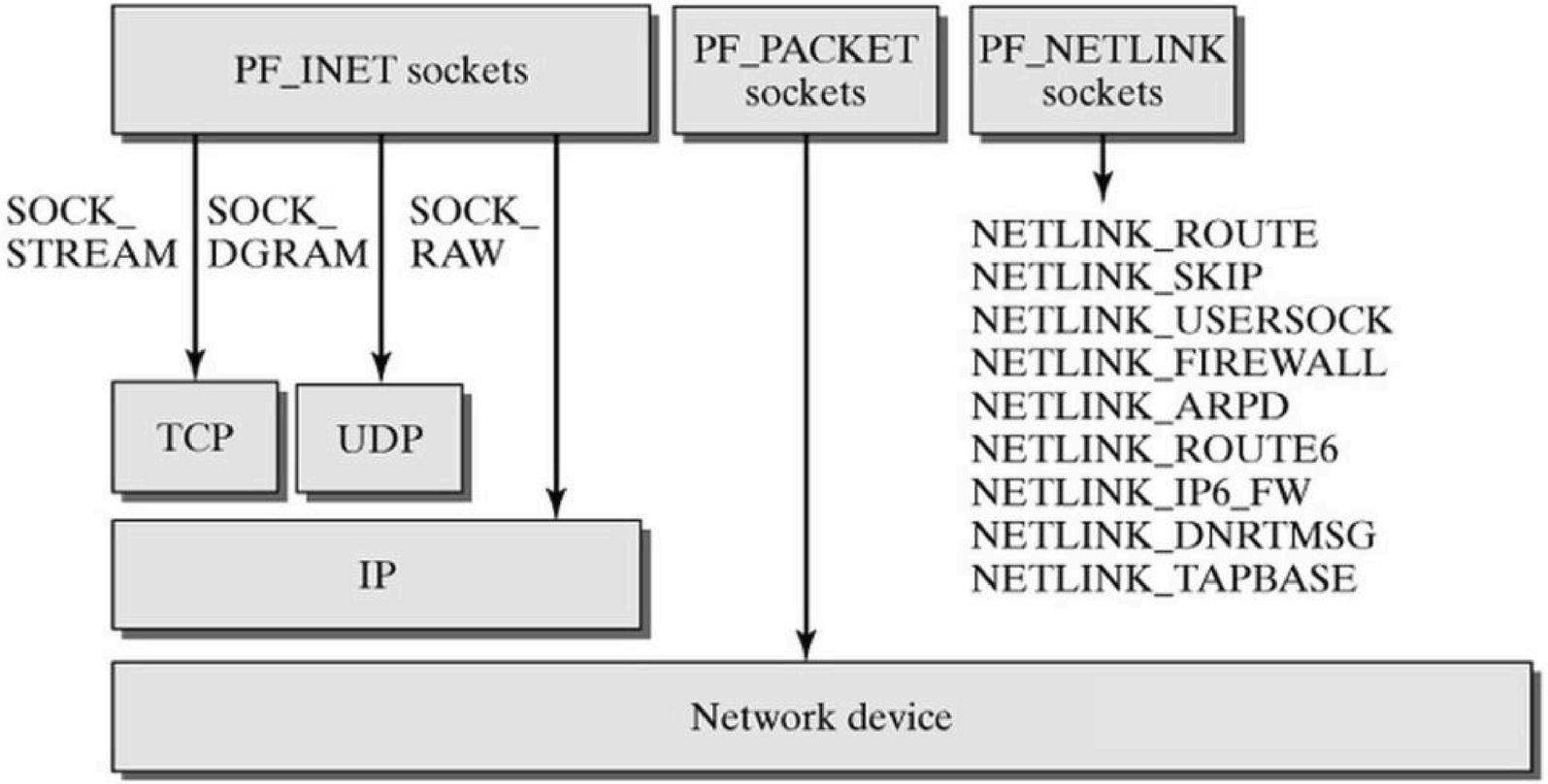
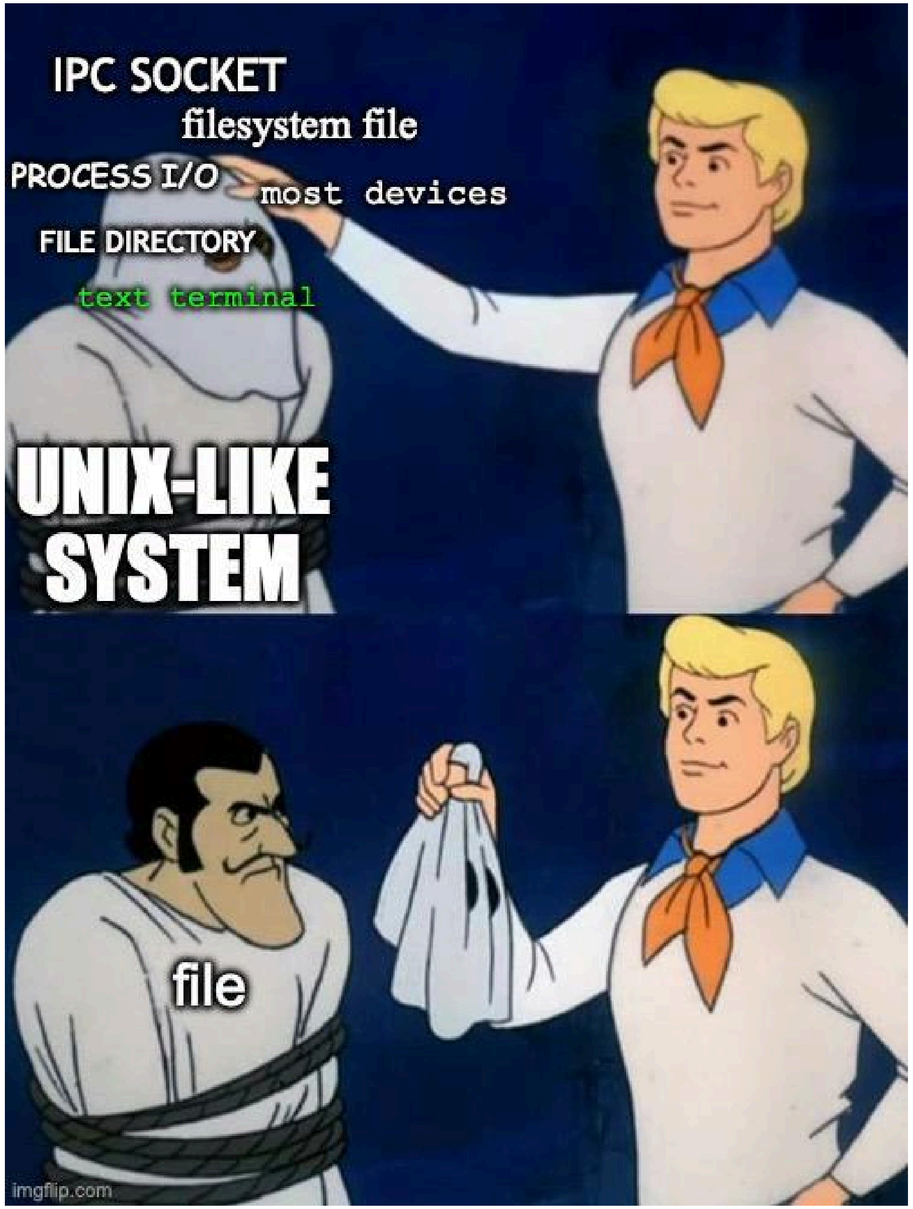
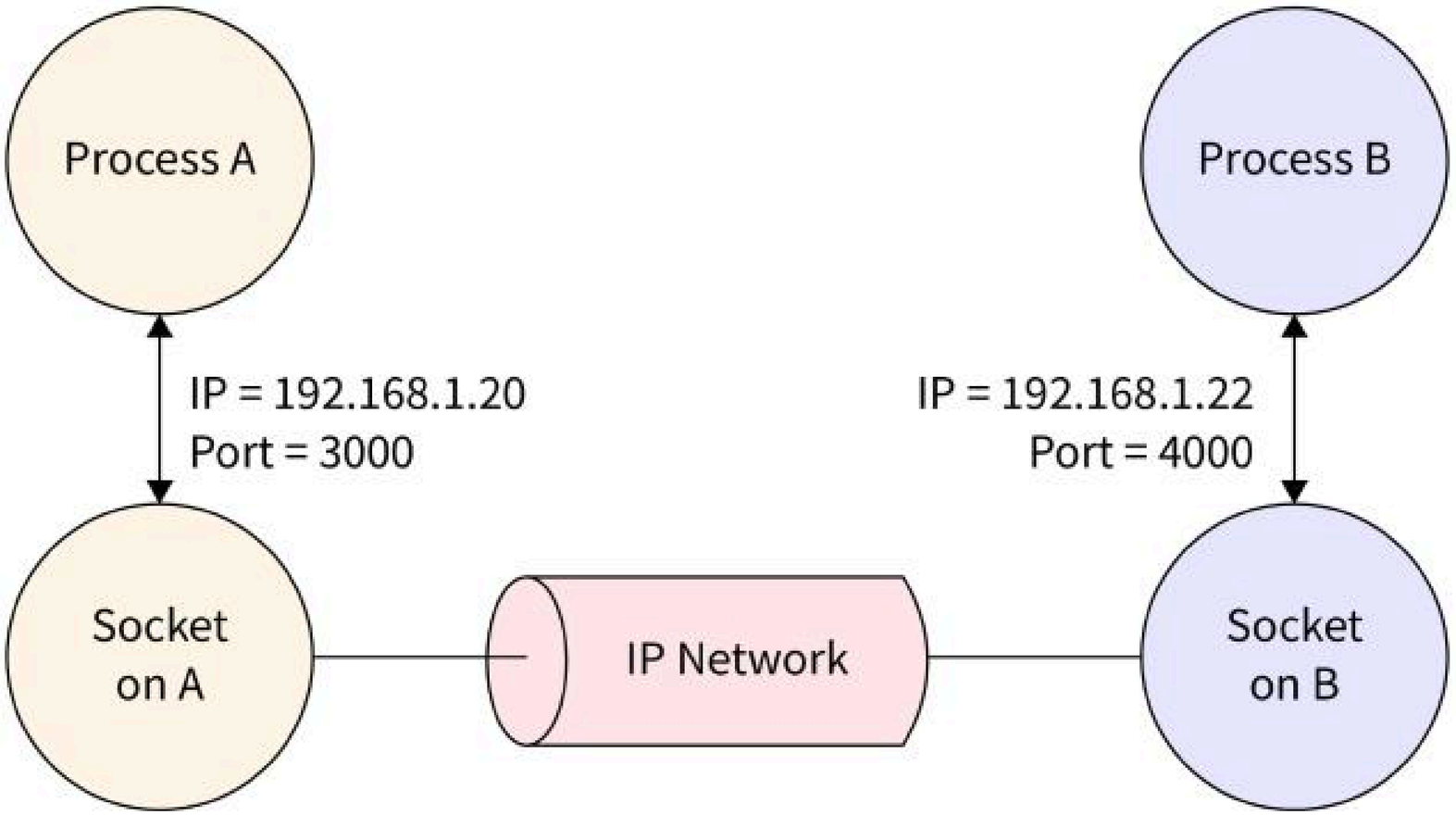
Сети в Unix-подобных системах



Модель OSI		Модель TCP/IP	
Прикладной уровень (application layer)	7	4	Прикладной уровень (application layer)
Уровень представления (presentation layer)	6		
Сеансовый уровень (session layer)	5		
Транспортный уровень (transport layer)	4	3	Транспортный уровень (transport layer)
Сетевой уровень (network layer)	3	2	Межсетевой уровень (internet layer)
Канальный уровень (data link layer)	2	1	Канальный уровень (link layer)
Физический уровень (physical layer)	1		


IPC SOCKET
filesystem file
PROCESS I/O
most devices
FILE DIRECTORY
text terminal
UNIX-LIKE SYSTEM
file
imgflip.com


Process A
Process B
IP = 192.168.1.20
Port = 3000
IP = 192.168.1.22
Port = 4000
Socket
on A
IP Network
Socket
on B

socket - create an endpoint for communication

LIBRARY top

Standard C library (*libc*, *-lc*)

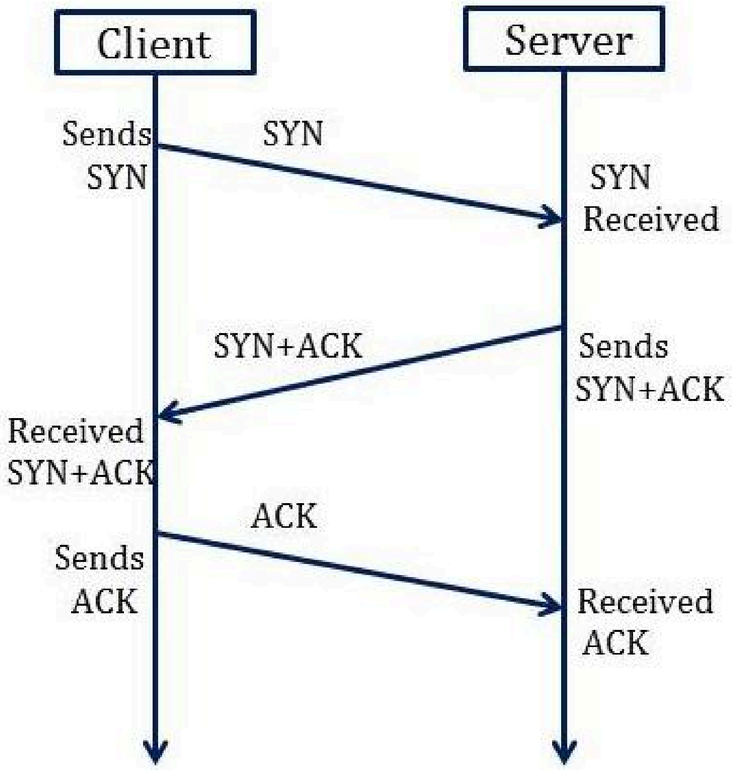
SYNOPSIS top

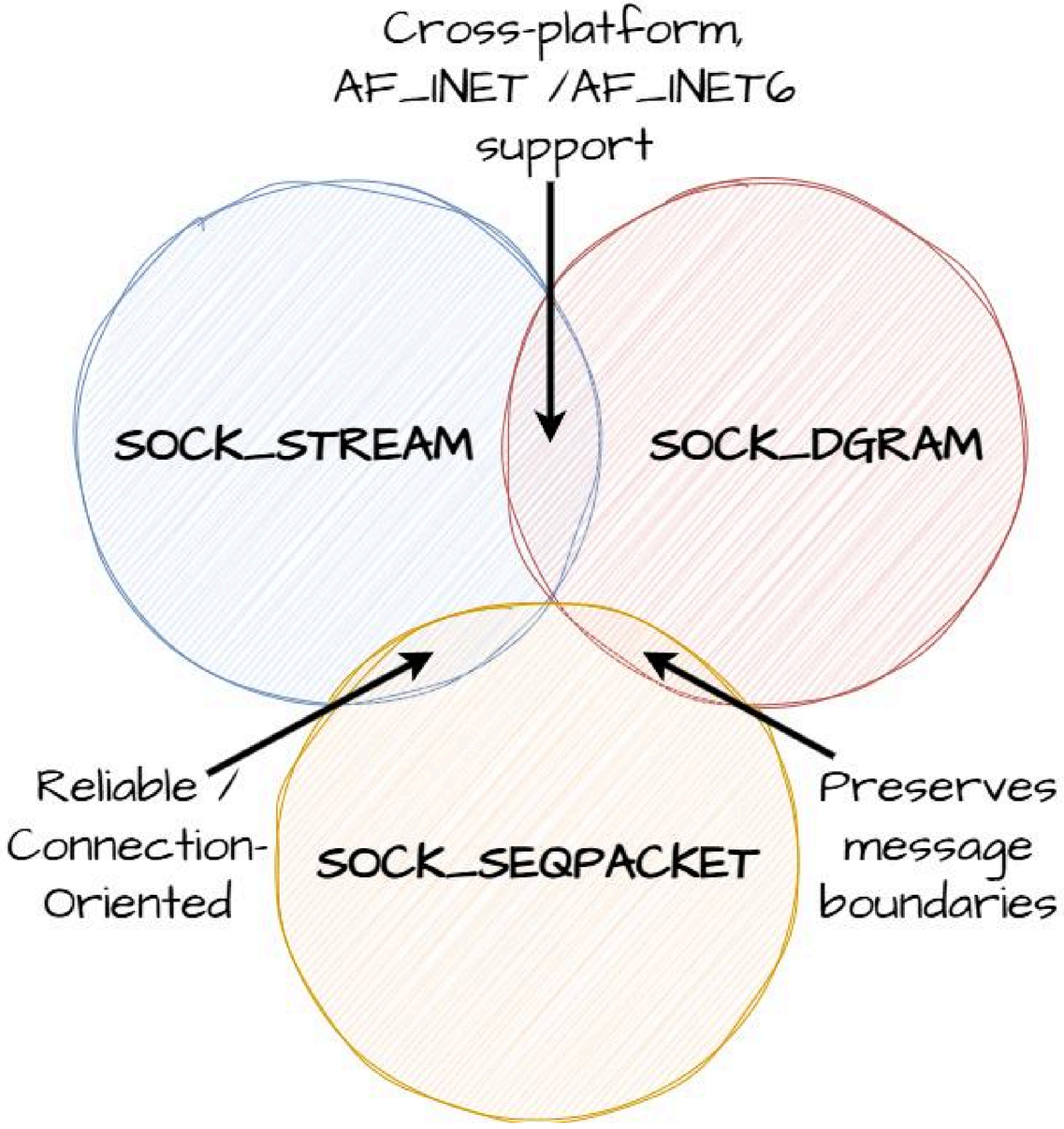
```
#include <sys/socket.h>

int socket(int domain, int type, int protocol);
```

DESCRIPTION top

socket() creates an endpoint for communication and returns a file descriptor that refers to that endpoint. The file descriptor returned by a successful call will be the lowest-numbered file descriptor not currently open for the process.


Client
Server
Sends SYN
SYN
SYN Received
SYN+ACK
Sends SYN+ACK
Received SYN+ACK
ACK
Sends ACK
Received ACK


Cross-platform, AF_INET /AF_INET6 support
SOCK_STREAM
SOCK_DGRAM
SOCK_SEQPACKET
Reliable / Connection-Oriented
Preserves message boundaries

SOCK_STREAM
Provides sequenced, reliable, two-way, connection-based byte streams. An out-of-band data transmission mechanism may be supported.

SOCK_DGRAM
Supports datagrams (connectionless, unreliable messages of a fixed maximum length).

SOCK_SEQPACKET
Provides a sequenced, reliable, two-way connection-based data transmission path for datagrams of fixed maximum length; a consumer is required to read an entire packet with each input system call.

SOCK_RAW
Provides raw network protocol access.

SOCK_RDM
Provides a reliable datagram layer that does not guarantee ordering.

SOCK_PACKET
Obsolete and should not be used in new programs; see packet(7).

Some socket types may not be implemented by all protocol families.

Name	Purpose	Man page
AF_UNIX	Local communication	unix(7)
AF_LOCAL	Synonym for **AF_UNIX**	
AF_INET	IPv4 Internet protocols	ip(7)
AF_AX25	Amateur radio AX.25 protocol	**ax25**(4)
AF_IPX	IPX - Novell protocols	
AF_APPLETALK	AppleTalk	ddp(7)
AF_X25	ITU-T X.25 / ISO/IEC 8208 protocol	x25(7)
AF_INET6	IPv6 Internet protocols	ipv6(7)
AF_DECnet	DECet protocol sockets	
AF_KEY	Key management protocol, originally developed for usage with IPsec	
AF_NETLINK	Kernel user interface device	netlink(7)
AF_PACKET	Low-level packet interface	packet(7)
AF_RDS	Reliable Datagram Sockets (RDS) protocol	**rds**(7) **rds-rdma**(7)
AF_PPPOX	Generic PPP transport layer, for setting up L2 tunnels (L2TP and PPPoE)	
AF_LLC	Logical link control (IEEE 802.2 LLC) protocol	
AF_IB	InfiniBand native addressing	
AF_MPLS	Multiprotocol Label Switching	
AF_CAN	Controller Area Network automotive bus protocol	
AF_TIPC	TIPC, "cluster domain sockets" protocol	
AF_BLUETOOTH	Bluetooth low-level socket protocol	
AF_ALG	Interface to kernel crypto API	
AF_VSOCK	VSOCK (originally "VMWare VSockets") protocol for hypervisor-guest communication	vsock(7)
AF_KCM	KCM (kernel connection multiplexer) interface	
AF_XDP	XDP (express data path) interface	

```
→ cat /etc/protocols
# Full data: /usr/share/iana-etc/protocol-numbers.iana

hopopt          0 HOPOPT
icmp            1 ICMP
igmp            2 IGMP
ggp             3 GGP
ipv4            4 IPv4
st              5 ST
tcp             6 TCP
cbt             7 CBT
egp             8 EGP
igp             9 IGP
bbn-rcc-mon    10 BBN-RCC-MON
nvp-ii         11 NVP-II
pup            12 PUP
emcon          14 EMCON
xnet           15 XNET
chaos          16 CHAOS
udp            17 UDP
mux            18 MUX
dcn-meas       19 DCN-MEAS
hmp            20 HMP
prm            21 PRM
xns-idp        22 XNS-IDP
trunk-1        23 TRUNK-1
trunk-2        24 TRUNK-2
leaf-1         25 LEAF-1
leaf-2         26 LEAF-2
rdp            27 RDP
irtp           28 IRTP
```

```c
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);

    if (sockfd == -1) {
        perror("socket");
        return 1;
    }

    printf("%d", sockfd);

    close(sockfd);
    return 0;
}
```

```c
#include <netinet/in.h>

struct sockaddr_in {
    sa_family_t     sin_family;     /* AF_INET */
    in_port_t       sin_port;       /* Port number */
    struct in_addr  sin_addr;       /* IPv4 address */
};
```

```c
struct sockaddr_in6 {
    sa_family_t     sin6_family;    /* AF_INET6 */
    in_port_t       sin6_port;      /* Port number */
    uint32_t        sin6_flowinfo;  /* IPv6 flow info */
    struct in6_addr sin6_addr;      /* IPv6 address */
    uint32_t        sin6_scope_id;  /* Set of interfaces for a scope */
};
```

```c
#include <sys/un.h>

struct sockaddr_un {
    sa_family_t     sun_family;     /* Address family */
    char            sun_path[];     /* Socket pathname */
};
```

1
1024
49151
65535
Общепринятые
HTTP - 80
HTTPS - 443
DNS - 53
...
Зарегистрированные
IANA
Для любых целей
ОС

```
INADDR_LOOPBACK (127.0.0.1)
INADDR_ANY (0.0.0.0)
inet_pton(AF_INET, "192.168.1.100", &addr.sin_addr);
```

htons() - 16

htonl() - 32

8080 = 1F90 → 901f = 36895

```c
#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>

int main() {
    struct sockaddr_in addr;

    memset(&addr, 0, sizeof(addr));

    addr.sin_family = AF_INET;
    addr.sin_port   = htons(8080);

    addr.sin_addr.s_addr = INADDR_ANY;

    inet_pton(AF_INET, "192.168.1.100", &addr.sin_addr);

    return 0;
}
```

1. `socket()` — Создание канала

Создает структуру данных в ядре и возвращает дескриптор (число)

2. `bind()`

Связывает сокет с конкретным «местом» (IP и портом).
Записывает в структуру сокета в ядре информацию о локальном адресе

3. `listen()` — Объявление о готовности

Переводит сокет в состояние ожидания входящих соединений
Создает в ядре очередь для входящих запросов (backlog). Сокет становится «пассивным»
Работает только для `SOCK_STREAM` (TCP)

4. `accept()` — Установление связи

Извлекает первый запрос из очереди ожидающих соединений.
Создает **новый** сокет специально для общения с этим конкретным клиентом. Изначальный сокет продолжает слушать других.
Если в очереди никого нет, программа «заснет» на этом вызове, пока кто-то не подключится.

5. `connect()` — Запрос на соединение

Используется клиентом, чтобы «позвонить» серверу.
Инициирует процедуру «трехстороннего рукопожатия» (TCP 3-way handshake).
После возврата функции соединение установлено и готово к передаче данных.

6. `send()` / `recv()` — Обмен данными

Специфические для сокетов версии `write()` и `read()`.
`send()`: Копирует данные из памяти программы в буфер ядра для отправки в сеть.
`recv()`: Забирает данные из буфера ядра, куда их положила сетевая карта.

7. `close()` — Уничтожение

Завершение работы с дескриптором.
Сообщает ядру, что ресурсы можно освободить. В TCP инициирует отправку пакета `FIN` для корректного закрытия сессии.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_len = sizeof(client_addr);
    char buffer[BUFFER_SIZE];
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == -1) {
        perror("-1 socket");
        exit(EXIT_FAILURE);
    }
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);
    if (bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr)) == -1) {
        perror("-1 bind");
        close(server_fd);
        exit(EXIT_FAILURE);
    }
    if (listen(server_fd, 5) == -1) {
        perror("-1 listen");
        close(server_fd);
        exit(EXIT_FAILURE);
    }
    printf("server port %d...\n", PORT);
    while (1) {
        client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
        if (client_fd == -1) {
            perror("-1 accept");
            continue;
        }
        printf("client_addr %s\n", inet_ntoa(client_addr.sin_addr));
        memset(buffer, 0, BUFFER_SIZE);
        ssize_t bytes_read = read(client_fd, buffer, BUFFER_SIZE - 1);
        if (bytes_read > 0) {
            printf("in: %s", buffer);
            write(client_fd, buffer, bytes_read);
        }
        close(client_fd);
        printf("close \n\n");
    }
    close(server_fd);
    return 0;
}
```

```
mars  ~/ /test  v15.2.1  12:10
→ ./socket
server port 8080...
client_addr 127.0.0.1
in: test1
close

client_addr 127.0.0.1
in: test2
close

client_addr 127.0.0.1
in: test3
close
```

```
mars  ~/ /test  v15.2.1  12:10
→ nc 127.0.0.1 8080
test1
test1

mars  ~/ /test  v15.2.1  12:10
→ nc 127.0.0.1 8080
test2
test2

mars  ~/ /test  v15.2.1  12:10
→ nc 127.0.0.1 8080
test3
test3

mars  ~/ /test  v15.2.1  12:10
→
```

epoll_create1 - создаёт epoll-instance и возвращает fd на него

- EPOLL_CLOEXEC, чтобы fd автоматически закрывался при exec()

epoll_ctl - управляет списком fd внутри epoll.

- добавить fd (EPOLL_CTL_ADD)
- изменить что слушать (EPOLL_CTL_MOD)
- удалить fd (EPOLL_CTL_DEL)

epoll_pwait - блокируется и ждёт пока хоть один fd из списка станет готов. Возвращает массив готовых событий и их количество, vожно передать таймаут

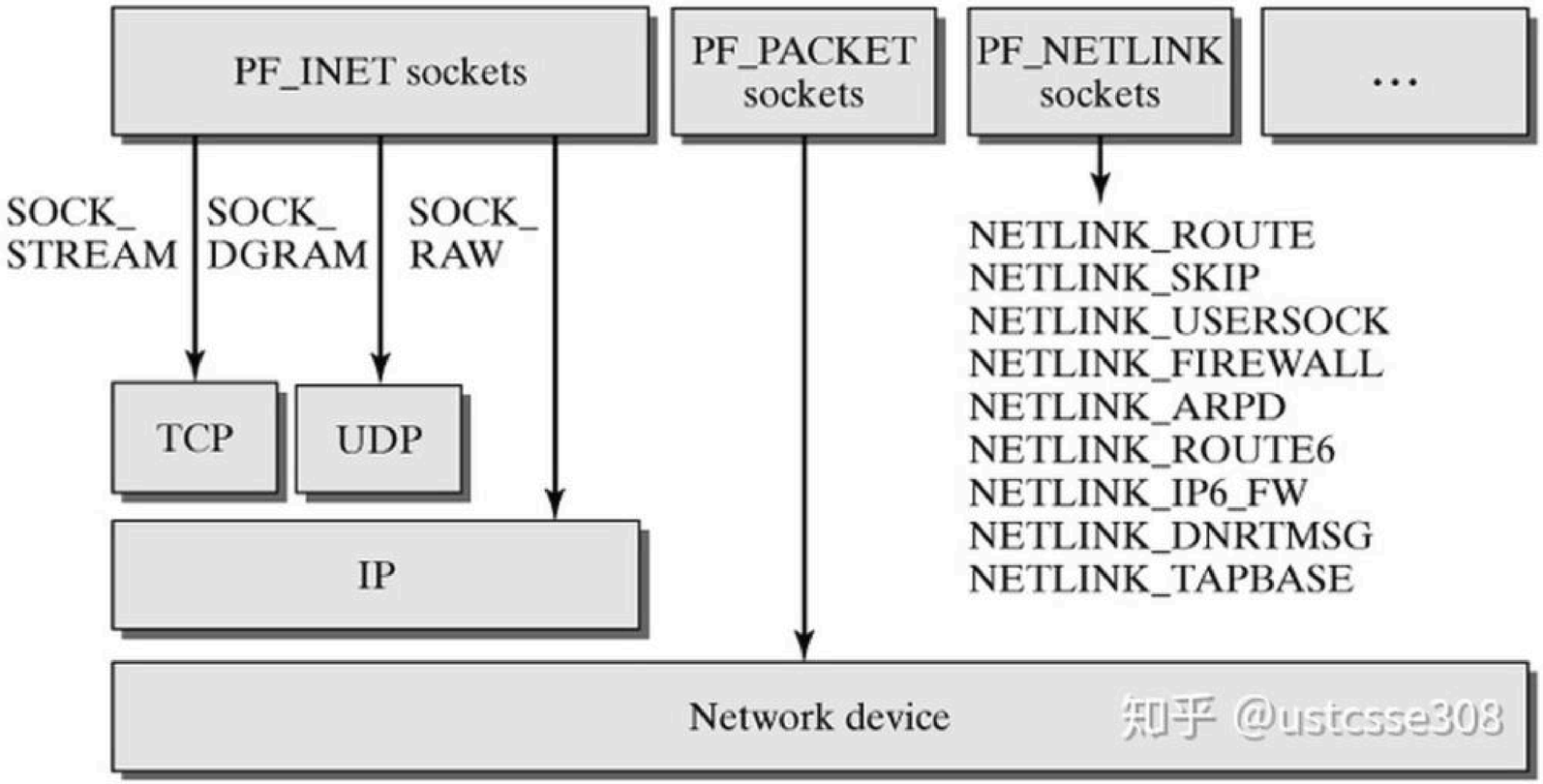
```
mars  ~/ /test  v15.2.1  12:25
→ sudo fuser -k 8080/tcp

mars  ~/ /test  v15.2.1  12:25
→ gcc -o socket socket.c

mars  ~/ /test  v15.2.1  12:25
→ ./socket
Server listening on 8080...
Client connected: 5
Recv: ls
Recv: whoami
```

```
mars  ~/ /test  v15.2.1  12:25
→ nc 127.0.0.1 8080
ls
ls
whoami
whoami
```

raw sockets


PF_INET sockets
PF_PACKET sockets
PF_NETLINK sockets
...
SOCK_ STREAM
SOCK_ DGRAM
SOCK_ RAW
NETLINK_ROUTE
NETLINK_SKIP
NETLINK_USERSOCK
NETLINK_FIREWALL
NETLINK_ARPD
NETLINK_ROUTE6
NETLINK_IP6_FW
NETLINK_DNRTMSG
NETLINK_TAPBASE
TCP
UDP
IP
Network device
知乎 @ustcsse308